CODE//CONNECT'26 · PROBLEM 01: DISASTER RESPONSE · ENGINEERING

System Design

PulseTriage: architecture, tech stack and building plan

How every part of PulseTriage is built and how the parts fit together: requirements, components, data, APIs, algorithms, security, reliability, testing, and the step-by-step plan for building it in about 36 hours with 4 people.

How to use this document

- **Everyone:** Part 1 (requirements) and Part 2 (architecture). Then the building plan in Part 14.
- **A, mobile:** Parts 4, 7.1–7.3 and 7.5, 9.7, 10. **B, server:** Parts 5, 7, 8, 9, 10, 11. **C, dashboard:** Parts 6 and 8. **D, data and AI:** Parts 5.3, 7.3, 9, 13 and the appendices.
- **Single source of truth:** every number lives in Appendix A (`shared/constants.json`). Every format lives in Part 7. If code disagrees with them, the code is wrong.
- Companion documents: the Team Brief (the why and the demo) and the online build plan (tasks and priorities).

Version 1 · 17 October 2026 · Numbers are demo defaults to tune; nothing here is medical advice.

Contents

1. Requirements
   - 1.1 Goal
   - 1.2 Users and actors
   - 1.3 Functional requirements
   - 1.4 Non-functional requirements
   - 1.5 Constraints and assumptions
   - 1.6 Out of scope
2. Architecture
   - 2.1 The big idea
   - 2.2 Components
   - 2.3 What runs where
   - 2.4 Key flows
   - 2.5 Decisions and trade-offs
3. Tech stack
4. Field app design
   - 4.1 Modules
   - 4.2 Screens
   - 4.3 Local storage
   - 4.4 Mesh protocol
   - 4.5 Connection modes
   - 4.6 Location pings
   - 4.7 Keeping the app alive
5. Server design
   - 5.1 Modules
   - 5.2 Ingest pipeline
   - 5.3 Read fast, refine later
   - 5.4 Scheduler jobs
   - 5.5 Outbound queue
   - 5.6 Storage and concurrency
   - 5.7 Caching
6. Dashboard design
7. Data design
   - 7.1 Wire format
   - 7.2 Packet kinds
   - 7.3 Hazard profiles
   - 7.4 Server database
   - 7.5 Phone database
8. API
9. Algorithms
10. Security and privacy
11. Reliability
12. Scale and performance
13. Testing
14. Building plan
    - 14.1 Repo layout
    - 14.2 Hour 0 setup
    - 14.3 Contracts first
    - 14.4 Phases and integration points
    - 14.5 Build order per person
    - 14.6 Definition of done
    - 14.7 Git workflow
    - 14.8 Demo runbook
15. What we'd revisit as it grows

Appendix A. Constants

Appendix B. Example profile: flood.json

PART 1

Requirements

1.1 Goal

Find the people who need help most in any disaster, warn them early, and get the right limited resources to them before it becomes critical, while working with no internet, unreliable phone networks and messy, incomplete data.

1.2 Users and actors

Actor	Uses	Needs from the system
Helper (resident, neighbour)	Field app	Get alerts; reply safe or need help; report for themselves or others; relay messages
Volunteer (boat, 4x4, nurse)	Field app	Everything a helper does; offer resources; receive welfare checks and jobs; send location
Rescue team	Field app	Receive jobs; report en route, arrived, done with an outcome; send location, fuel, battery
Dispatcher	Dashboard	Issue alerts; see ranked cases with reasons; assign teams; track teams and supplies; override with a reason
Call-taker	Dashboard	Type in reports from calls, radio and walk-ins; paste forwarded messages
Shelter manager	Dashboard or phone call	Update head count and stock
Person with no phone	Nobody's device	Be warned (door knock, loudspeaker) and be counted (neighbour reports, registry, Silent Zones)

1.3 Functional requirements

ID	The system must	Priority
FR-1	Let a dispatcher issue a signed alert for chosen zones, severity and disaster type, from a template, in local languages	Must
FR-2	Deliver alerts to Field phones phone to phone with no internet, and show them full screen with read-aloud	Must
FR-3	Collect replies to alerts and show warning coverage per zone; turn unconfirmed registered people into welfare checks	Should
FR-4	Accept reports from the app, alert replies, the call-taker form, the paste box and (stretch) text messages	Must
FR-5	Upload packets straight to the server whenever the phone can reach it (Wi-Fi or mobile data); relay them phone to phone (store and forward) only when it can't	Must
FR-6	Read free text into structured fields with an AI reader, falling back to keyword rules offline	Must
FR-7	Merge duplicate reports into cases and keep a confidence score for every case	Must

ID	The system must	Priority
FR-8	Rank cases with visible priority points driven by hazard profiles; show the reasons	Must
FR-9	Detect Silent Zones from expected vs actual reports and warning coverage	Must
FR-10	Suggest the fastest free capable team and the nearest place with space; show the trade-off; a person confirms	Must
FR-11	Send jobs to team phones and receive en route, arrived, done and outcome	Must
FR-12	Track every team's status, last position and arrival; flag overdue and not-heard-from teams	Must
FR-13	Track supplies at the command post, shelters and on teams; warn on low stock; forecast run-out	Must / Should
FR-14	Track shelters and cooling centres: capacity and head count	Should
FR-15	Support any disaster type through hazard profile files, with flood and heatwave fully tested	Must
FR-16	Log every dispatcher action and override with a reason	Must

1.4 Non-functional requirements

Quality	Target	How we meet it
Works offline	Every Must feature works with no internet	Laptop server, own router, mesh, keyword reader, offline map tiles
Latency	Packet processed in under 50 ms; dashboard refresh every 2 s; alert queued in under 1 s	Single process, SQLite WAL, in-memory snapshot cache
Speed when connected	A report from a phone with a connection is on the dashboard within 2 s	Direct upload; the mesh is only a fallback
No data loss	A report is never lost once saved on a phone	Phones keep packets until the server acknowledges; server dedupes by ID; database backups
Trust and explainability	Every rank can be explained line by line	Rules with visible points; the AI never ranks
Security	Nobody can fake an alert; relays can't read private data	Ed25519 signatures, libsodium sealed boxes, signed alerts
Privacy	Minimum data; deleted after the crisis	Opt-in registry; geohash-5 outside the seal; retention script
Accessibility	Usable by people who can't read well or see well	Read-aloud, big buttons, icons, local languages
Battery	A team phone lasts a full shift	Pings every 2 min on jobs, 10 min idle; short mesh sessions; power banks
Simplicity	Buildable by 4 people in 36 hours	Few files, no extra frameworks, one database file

1.5 Constraints and assumptions

- 4 people, about 36 hours, AI coding tools for everyone.
- 3 Android phones with Google Play services (plus 1 spare), 1 Windows laptop, 1 Wi-Fi router or hotspot.

- No guaranteed internet at the venue; the demo must run with it switched off.
- Android only (Nearby Connections). Phones run the app in the foreground during the demo.
- All demo data is invented; one demo town with 8 to 10 zones.

1.6 Out of scope for the hackathon

- iPhone support, multiple command posts syncing, user accounts and login for the dashboard.
- Real road routing, live hazard feeds, cell broadcast, mass text messages and robocalls (stretch at most).
- Production key storage (Android Keystore, hardware security modules) and audited cryptography.

PART 2

Architecture

2.1 The big idea

Everything that moves is a **signed packet**: reports, alerts, replies, jobs, acknowledgements and team pings. A phone that can reach the server (over the command-post Wi-Fi, home Wi-Fi or mobile data) sends and fetches packets **directly**. Only a phone that can't reach it switches to the **mesh**: it stores packets and passes them to nearby phones (store and forward), preferring phones that are online, until one of them uploads them. The laptop runs one server that turns packets into cases, scores them with rules from **hazard profiles**, and serves the dashboard. Because there is one protocol and one pipeline, every channel and every disaster type goes through the same code.

2.2 Components

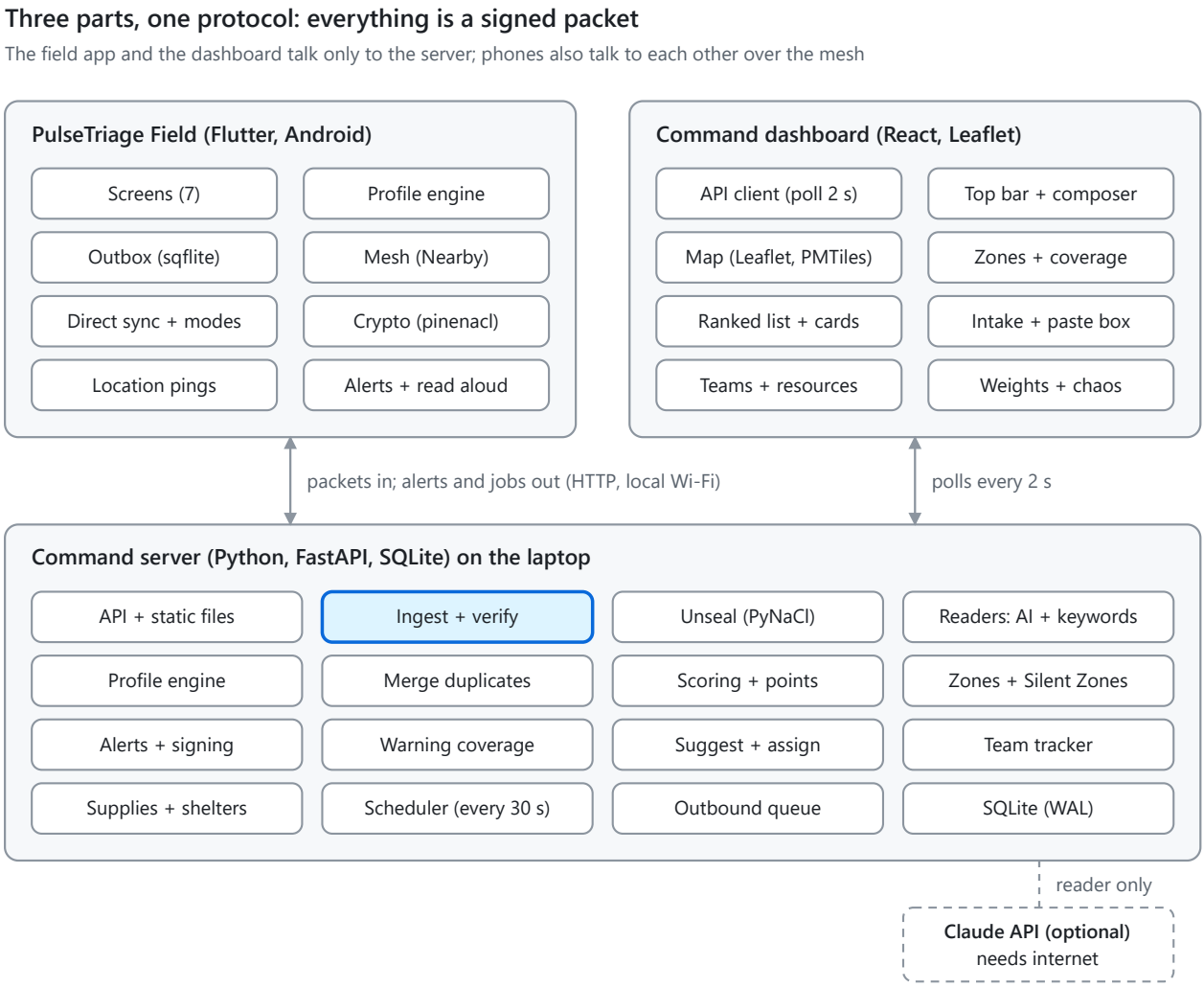


Figure 1. Component diagram. The highlighted module is the entry point of the server pipeline: every packet, from any channel, goes through ingest and verify.

Part	Responsibility	Talks to
Field app	Alerts, report form, welfare checks, jobs, location pings, outbox, connection modes, direct sync, mesh relay when offline, crypto	Server (HTTP, whenever reachable); other phones (Nearby) only when offline
Command server	Ingest, verify, unseal, read, merge, score, zones, coverage, alerts, suggestions, team tracker, supplies, outbound queue, storage	Phones (directly, or through online phones), dashboard, Claude API (optional)
Dashboard	Show and command: map, ranked list, cards, alert composer, intake, paste, teams and resources, weights, chaos	Server only (HTTP JSON)
Scenario tools	Generate demo packs, replay them, run the comparison simulation	Server API (replay), files

2.3 What runs where

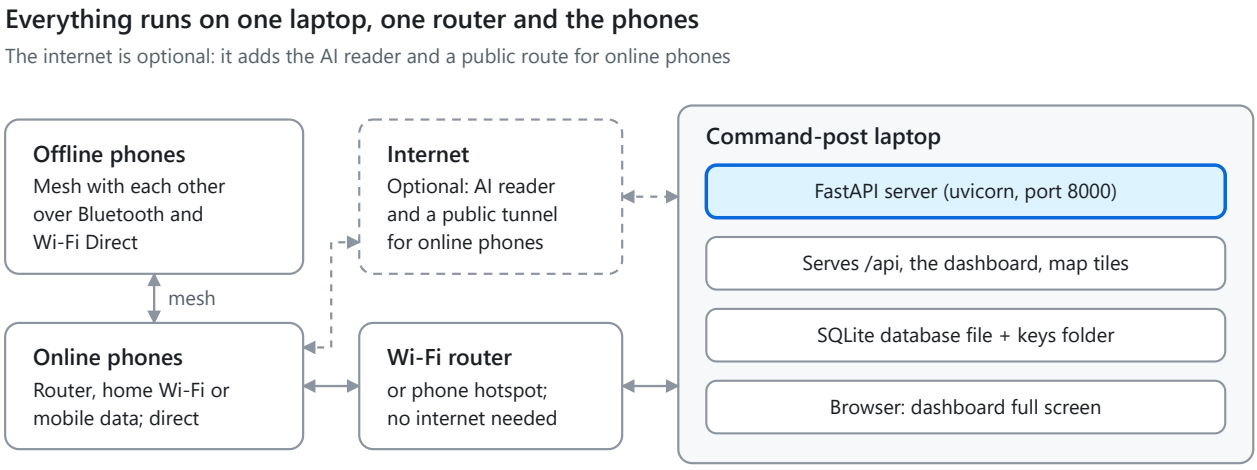


Figure 2. Deployment. The dashboard is a static web app served by the same FastAPI process, so the whole command post is one program and one URL.

2.4 Key flows

A report: from a phone with no signal to the ranked list

Read fast with keywords, refine later with the AI; nothing waits on the internet

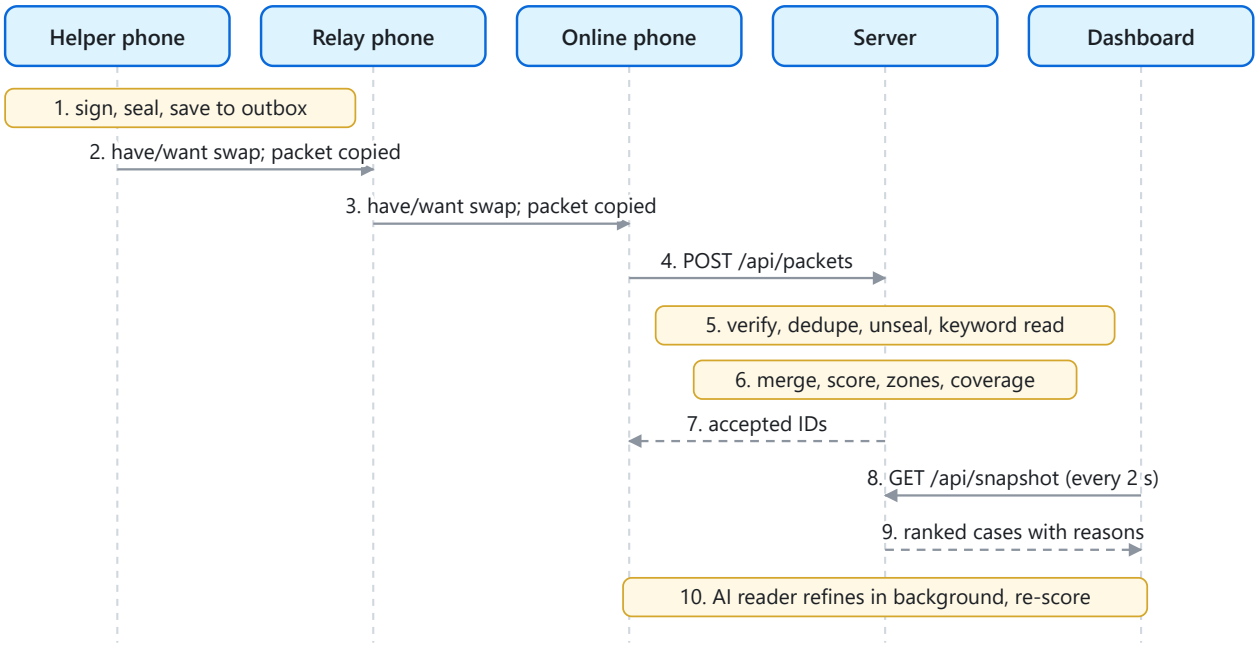


Figure 3. Report flow for a helper with no connection. With a connection, the helper phone uploads directly (step 4) and steps 2 and 3 never happen. Steps 5 and 6 take milliseconds; step 10 happens later and only when the internet is available.

An alert: out through the mesh, replies back into coverage

Alerts are urgency 4, so every phone sends them before anything else

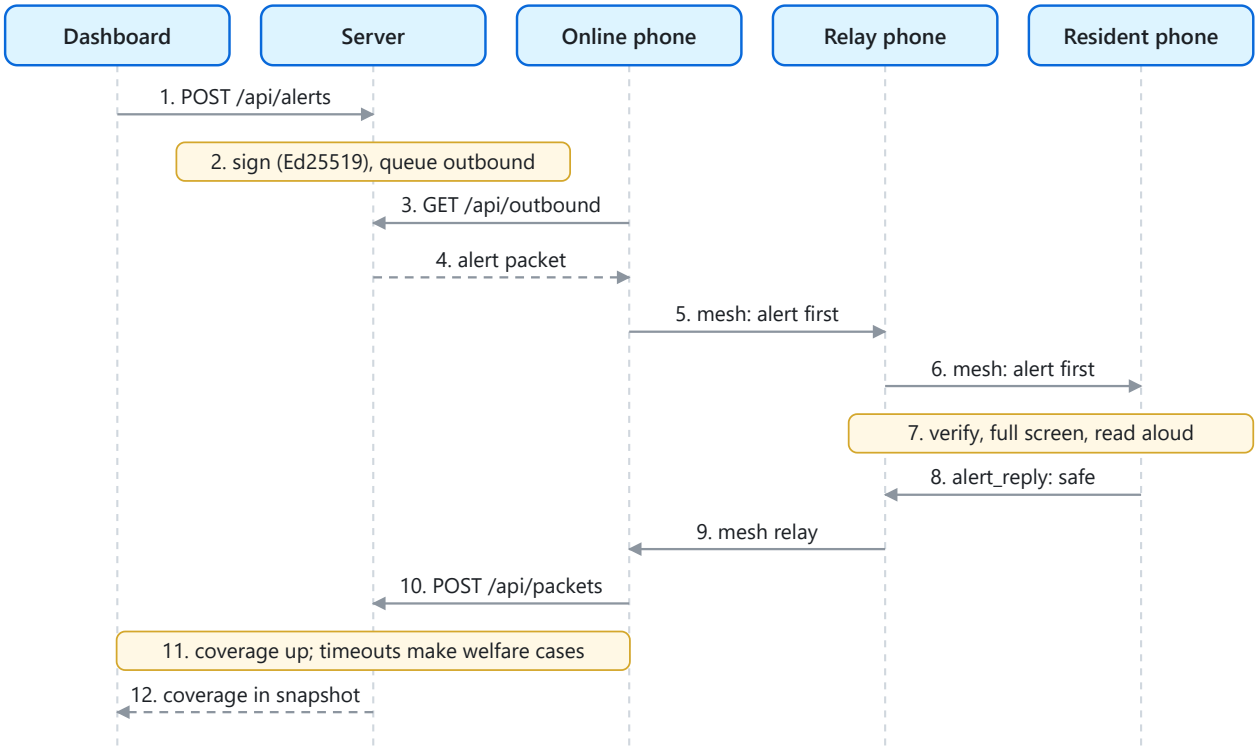


Figure 4. Alert flow. Phones ignore any alert whose signature does not match the command post key saved at setup.

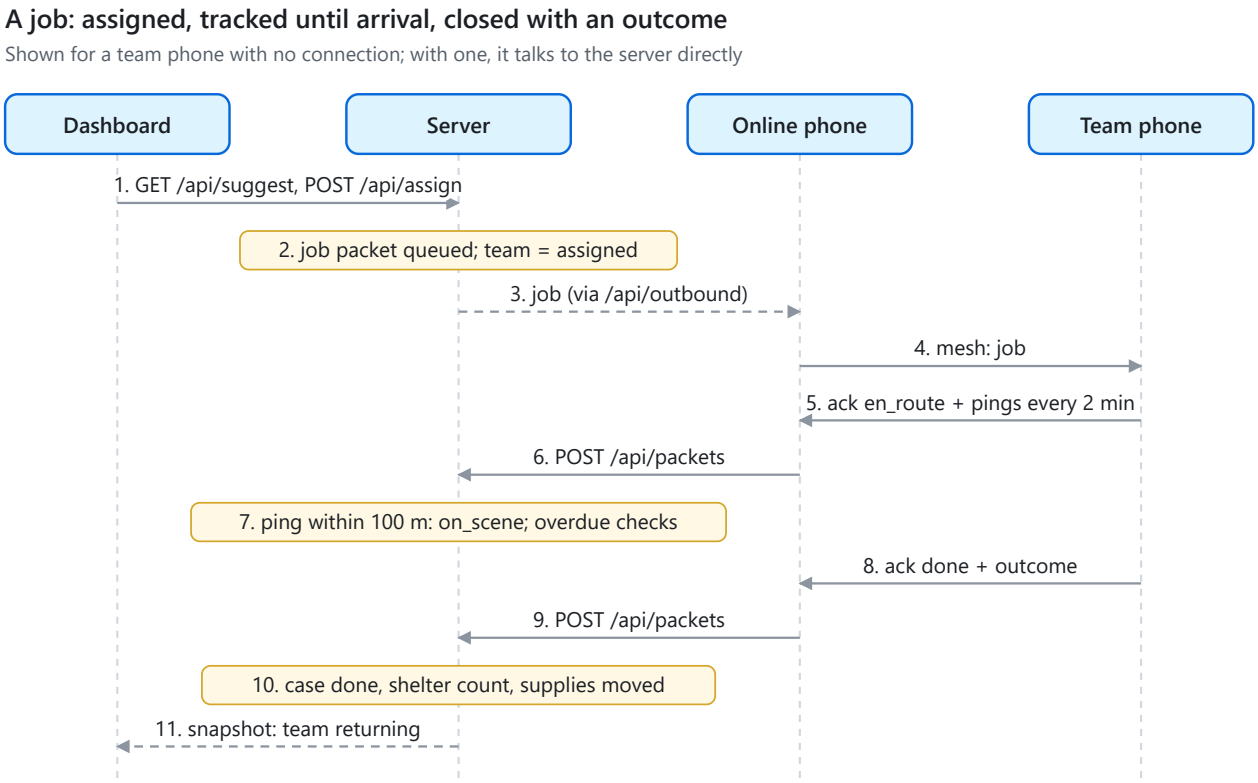


Figure 5. Job and tracking flow. Arrival is detected from pings; the outcome updates the case, the shelter and the supplies in one transaction.

2.5 Decisions and trade-offs

Decision	Chosen	Alternatives	Why	Revisit when
Field platform	Flutter, Android only	Kotlin native; React Native; web app	One codebase; ready Nearby plugin; AI tools write it well	iPhone support is needed
Phone to phone	Nearby Connections, P2P_CLUSTER	Custom Bluetooth LE; Wi-Fi Aware; LoRa radios	Works offline, handles Bluetooth and Wi-Fi for us	Areas are too big: add LoRa gateways
When to use the mesh	Only when the server can't be reached	Always relay through the mesh	Direct upload is faster, more reliable and saves battery	Never: the mesh stays as the fallback
Sync protocol	Store and forward with have/want lists	Flood everything; CRDTs	Simple, sends only what is missing	Thousands of packets per phone: use Bloom filters
Server	One FastAPI process + SQLite file	Node; Postgres; cloud backend	Runs on a laptop with no internet; nothing to install	Several command posts or many dashboard users
Dashboard updates	Poll a snapshot every 2 s	WebSockets; server-sent events	Simplest and robust to restarts	Many dashboards or large case counts
Ranking	Visible rules from profiles	Machine learning	Explainable, tunable, works offline	Real dispatch data exists to validate

Decision	Chosen	Alternatives	Why	Revisit when
AI	Claude Haiku, read fast then refine	Local model; AI-only reading	Quality when online; never blocks	Fully offline AI is required
Crypto	Ed25519 signatures + libsodium sealed boxes	TLS only; PGP	Protects each message through untrusted relays	Key rotation and revocation at scale
Signed body as a string	Sign the exact JSON text	Canonical JSON on both sides	Avoids Dart vs Python serialisation mismatches	Never: it is simply safer
Maps	Leaflet + one PMTiles file	Google Maps; Mapbox	Offline and free	Need live traffic or routing

PART 3

Tech stack

Pin exact versions in hour 0 (`pubspec.lock`, `requirements.txt` with `==`, `package-lock.json`) and never upgrade during the hackathon.

Field app

Layer	Choice	What we use it for
Language and UI	Flutter (Dart), Android	All screens; plain StatefulWidgets plus small service classes
Phone to phone	`nearby_connections`	Advertise, discover, connect, send byte payloads
Permissions	`permission_handler`	Bluetooth, nearby Wi-Fi devices, location at runtime
Storage	`sqflite` + `path`	Outbox and relay store, settings, seen alerts
Location	`geolocator`	GPS for reports and team pings
Read aloud	`flutter_tts`	Speak alerts offline with installed voices
QR	`qr_flutter`, `mobile_scanner`	Setup QR scan; QR backup show and scan
Crypto	`pinenacl`	Ed25519 signing, sealed boxes to the command post
IDs and HTTP	`uuid`, `http`	Packet IDs; direct sync with the server
Network changes	`connectivity_plus`	Notices Wi-Fi or mobile data coming and going; triggers a server check

Server

Layer	Choice	What we use it for
Language and web	Python 3.11+, FastAPI, Uvicorn, Pydantic	API, static files for the dashboard and tiles
Storage	SQLite (standard library `sqlite3`), WAL mode	One file database
Crypto	PyNaCl	Verify signatures, open sealed boxes, sign alerts
AI	`anthropic` SDK, model `claude-haiku-5-5`	Message reader, alert translation help
Geometry	`shapely`	Point in zone polygon, distances (with a haversine helper)
Setup QR	`qrcode` + `pillow`	PNG of the setup QR
Tests	`pytest`	Unit and replay tests
Public route	`cloudflared` quick tunnel	Gives the laptop a public HTTPS address when it has internet, so online phones anywhere can upload

Dashboard

Layer	Choice	What we use it for
Framework	React + Vite (JavaScript)	Single-page app built to static files
Map	`leaflet`, `react-leaflet`, `protomaps-leaflet`	Offline map from a .pmtiles file, zones, pins, team icons
Styling	Plain CSS	Big, projector-friendly layout

Data and tooling

Layer	Choice	What we use it for
Scenario tools	Python scripts + `matplotlib`	Generate packs, replay, simulation chart
Map data	Protomaps extract (`.pmtiles`), geojson.io	Offline base map; zone polygons
Everything else	Git + GitHub, Android Studio, VS Code or Cursor, curl	Source control, phone builds, editing, API checks

PART 4

Field app design

4.1 Modules

Module (file)	Responsibility
`models/packet.dart`	Wire packet, body fields, kinds, urgency; parse and build
`services/store.dart`	SQLite tables (7.5); save, list by priority, mark delivered, evict expired, replace by `replace_key`
`services/crypto_service.dart`	Device Ed25519 keys; sign body; verify signatures; seal to the command post box key
`services/mesh_service.dart`	Nearby advertise and discover, connection lifecycle, have/want/packets protocol (4.4)
`services/connection_service.dart`	Online or offline (4.5): server checks, direct upload and download, mesh discovery on or off
`services/profile_service.dart`	Load hazard profiles from assets; evaluate rules for urgency hints and questions
`services/location_service.dart`	GPS for reports; team pings on a timer (4.6); arrival check
`services/alert_service.dart`	Verify, show full screen, sound, vibrate, read aloud, record reply
`screens/*.dart`	Setup, Home, Alert, Report form, Welfare check, Jobs, Mesh and QR

4.2 Screens

Screen	Key elements	Produces
Setup	Role, name, languages, skills; scan setup QR (server URLs, command post keys, server time)	Settings; device keys; `volunteer` packet if skills
Home	Latest alert; big red report button; status line (phones nearby, waiting, last sync)	Navigation
Alert	Severity band, headline, 3 steps, place, Read aloud, three reply buttons, pass it on, tips	`alert_reply`
Report form	What's happening, for whom, people, conditions, profile questions, note, landmark, GPS	`report`
Welfare check	Who or where; Safe or Needs help; outreach list for volunteers	`welfare_check` (+ `report`)
Jobs	Assigned jobs; On my way, Arrived, Done (outcome form), Can't reach; Out of service, Ready; fuel	`ack`
Mesh and QR	Peers, packet counts, Show QR, Scan QR, relay switch	Imported packets

4.3 Local storage

One SQLite file per phone (schema in 7.5). Rules: never delete a packet the phone made until the server has acknowledged it; drop relayed packets after they expire; cap the store at 2,000 packets, evicting the oldest low-urgency relayed packets first; keep only the newest packet per `replace_key` (team pings, alert updates).

4.4 Mesh protocol

Nearby Connections with strategy `P2P_CLUSTER` and service ID `pulsetriage`. The mesh is a fallback (4.5): offline phones advertise and discover at the same time; online phones only advertise, so offline neighbours can find them. Messages are UTF-8 JSON sent as byte payloads:

```
{"t": "hello", "device": "dev_4KQ2", "role": "helper", "v": 1}
{"t": "have",  "ids": ["8f1c...", "a93b...", ...]}        // valid, unexpired packets only
{"t": "want",  "ids": ["a93b..."]}                         // the ones I don't have
{"t": "packets", "items": [ <wire packet>, ... ]}          // batches under 30 KB
{"t": "done"}                                              // I have sent everything you wanted
```

```
on_connected(peer):
    send(hello)
    send(have(ids_of(store.valid_packets())))

on_message(peer, m):
    if m.t == "have":
        send(want([i for i in m.ids if not store.has(i)]))
    if m.t == "want":
        for batch in batches(store.sorted_by_priority(m.ids), 30_000):
            send(packets(batch))
        send(done)
    if m.t == "packets":
        for p in m.items:
            if verify(p) and not expired(p) and p.hops < p.max_hops and not store.has(p.id):
                store.save(p, hops=p.hops + 1)     # replaces older packet, same replace_key
                if p.kind == "alert":
                    alert_service.show(p)
    if both sides sent done:
        disconnect(peer); cooldown(peer, 15 s)

# send order: urgency desc, then created_at desc
# alert 4, revoke 4, job 3, need-help reply 3, report 0-3, ack 2, team ping 1
```

4.5 Connection modes: direct first, mesh only as a fallback

A phone that can reach the server sends everything straight to it. Phone-to-phone relay is used only when it can't. "Reach the server" means a real answer from `/api/status`, not just "Wi-Fi is on": Wi-Fi with no route to the server counts as offline, and mobile data that reaches the server counts as online.

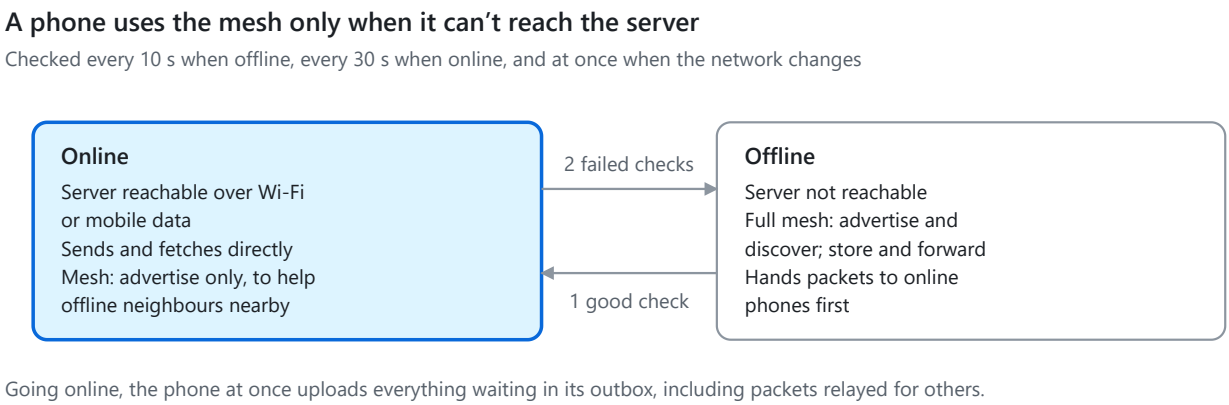


Figure 6. Connection modes. The phone switches on its own; the user never has to choose.

Mode	Its own packets	Alerts and jobs	Mesh
Online	Uploaded at once with `POST /api/packets` (up to 100 per request); retried every 10 s until accepted	`GET /api/outbound` every 10 s	Advertise only, no discovery. Offline phones that connect get a normal swap: their packets are uploaded at once; they receive alerts and jobs they lack
Offline	Saved to the outbox and relayed through the mesh	Arrive through the mesh	Full mesh (4.4): advertise and discover; connect to online phones first

Switching

- **Checks:** `GET {server}/api/status` with a 2 s timeout, trying each server address from the setup QR in order (local first, then public). Every 10 s when offline, every 30 s when online, and at once when Android reports a network change (`connectivity_plus`).
- **Offline to online:** after 1 good check. Stop discovery, keep advertising as online, upload the whole outbox (own and relayed packets), then fetch outbound.
- **Online to offline:** after 2 failed checks in a row, or a failed upload followed by a failed check. Start discovery. Nothing is lost: packets stay in the outbox until the server accepts them.
- **No double work:** packets the server has accepted are marked delivered and never pushed into the mesh. Copies already handed to neighbours are harmless: the server drops duplicates by ID.
- **Errors:** back off 10, 20, then 30 s; the download cursor means nothing is missed.

How offline phones find online ones

The Nearby advertising name carries the mode, for example `PT1|on|dev_4KQ2` or `PT1|off|dev_9ZZ1` . Offline phones connect to `on` phones first, because one hop to an online phone reaches the server. A setting, **Help offline neighbours** (on by default), lets an online phone keep advertising; switching it off stops all phone-to-phone traffic while online.

Reaching the server from anywhere

- **Local address:** the laptop on the command-post router, for example `http://192.168.0.10:8000` . Works with no internet.
- **Public address:** when the laptop has internet, `cloudflared tunnel --url http://localhost:8000` gives it a public HTTPS address (a free quick tunnel on trycloudflare.com), so phones on home Wi-Fi or mobile data

anywhere upload directly. The address changes each run, so phones rescan the setup QR. Production: a fixed tunnel or a small cloud relay.

- Packets are signed and sealed, so the public route carries nothing a stranger can read or change.

4.6 Location pings

- Rescue-team and volunteer roles only. Every 2 min while a job is open, every 10 min when idle. Sent directly when online; through the mesh when offline.
- Ping = sealed `team_ping` with position, accuracy, the time the position was taken, status, case ID, battery and fuel; `replace_key = "ping:<team_id>"`.
- The phone also checks arrival itself: within 100 m of the job's location it sends `ack arrived` (the server checks too).

4.7 Keeping the app alive

Android pauses Bluetooth discovery and timers when apps go to the background. For the demo: keep the app in the foreground with the screen on (a wakelock flag on the main screen), battery saver off. A foreground service with a persistent notification is the production answer and a stretch goal.

PART 5

Server design

5.1 Modules

File	Responsibility
`app/main.py`	FastAPI app, routes, static files (`/` dashboard, `/tiles`), lifespan starts the scheduler
`app/config.py`	Loads `shared/constants.json` , paths, environment (`ANTHROPIC_API_KEY`)
`app/db.py`	SQLite connection (WAL), schema creation, small query helpers, one write lock
`app/crypto.py`	Load command post keys; verify, unseal, sign; make setup QR payload
`app/ingest.py`	The pipeline in 5.2; dispatch by packet kind
`app/profiles.py`	Load profiles; rule matching (`all` tokens)
`app/reader.py`	Keyword reader (instant) and AI reader (background, cached)
`app/merge.py` , `scoring.py`	Duplicate merging; hours left, points, reasons, confidence
`app/zones.py` , `coverage.py`	Hard-hit zones, expected vs actual, Silent flag; warning coverage and timeouts
`app/alerts.py`	Create, sign, queue, update and cancel alerts; announcement scripts
`app/suggest.py`	Capable free teams, travel times, trade-off, assign and job packets
`app/teams.py` , `supplies.py`	Team tracker (status, pings, arrival, overdue); inventory, moves, burn rate
`app/scheduler.py`	Background loop every 30 s (5.4)
`app/snapshot.py`	Builds the cached dashboard snapshot
`scripts/make_keys.py` , `reset_demo.py` , `backup.py`	Key generation, demo reset per pack, database backups

5.2 Ingest pipeline

Every packet from every channel (phone upload, intake form, paste box, replay) goes through the same function. Intake and paste build an unsigned internal packet with `source` set, and skip steps 3 and 6.

```
def ingest(wire, received_at):
    body = parse_json(wire.body)                      # 1. size <= 8 KB, valid JSON, known kind
    if db.packet_exists(body.id):                     # 2. same ID twice is fine (idempotent)
        return "duplicate"
    verify_ed25519(body.pubkey, wire.body, wire.sig)  # 3. reject: "bad_signature"
    device = devices.get_or_register(body.from_device, body.pubkey)   # pubkey changed: reject
    check_expiry(body, received_at)                   # 4. allow 1 h of clock skew
    check_rate_limit(device)
    db.save_raw(wire, body, received_at)              # 5. raw log first
    payload = unseal(body.sealed) if body.sealed else body.public     # 6.
    with db.write_lock():                             # 7. dispatch in one transaction
        match body.kind:
            case "report":
                r = reports.add(payload)
                reader.keywords(r); merge.attach(r); scoring.rescore(r.case)
            case "alert_reply":   coverage.record(payload)
            case "welfare_check": coverage.record_check(payload)    # may create a report
            case "volunteer":     teams.upsert_volunteer(payload)
            case "ack":           teams.on_ack(payload)     # en_route, arrived, done, ...
            case "team_ping":     teams.on_ping(payload)
        zones.mark_dirty(); snapshot.mark_dirty(); audit.log(body)
    if body.kind == "report" and ai_online():
        reader.enqueue_ai(r)                          # 8. refine later (5.3)
    return "accepted"
```

5.3 Read fast, refine later

- The keyword reader runs inside ingest, so every report is ranked within milliseconds, online or offline.
- If the AI is reachable, the report goes on an in-memory queue. A background worker (4 at a time) calls Claude with a 4 s timeout, caches the answer by a hash of the text, updates the report (`reader = "ai"`) and re-scores its case.
- If the AI disagrees with the keywords, the AI wins for detail fields; the case card shows "read by AI" or "read by keyword rules (AI offline)".
- AI health is checked every 60 s with a tiny request; the dashboard banner follows it.

5.4 Scheduler jobs (every 30 s)

Job	What it does
Re-score	Hours left shrink as time passes, so recompute points for every open case (cheap: a few thousand rows)
Stale and re-check	Confidence decay; "re-check" badge after 3 h without an update
Coverage timeouts	Unconfirmed registered people become "unconfirmed" cases after 30 min (emergency) or 2 h (warning)
Zones	Recompute expected, actual, coverage and the Silent flag when dirty
Team checks	Overdue (ETA + 50%, at least 10 min) and not heard from (15 min without a ping while en route)
Supplies	Burn rate over the last 3 h; low-stock warnings
Alerts	Expire alerts past `valid_until` ; queue updates

Job	What it does
Backup	Every 5 min copy the database to `backups/` (SQLite backup API)

5.5 Outbound queue

Packets for the mesh (alerts, jobs, revocations) are rows in `outbound` with an increasing `seq`. Each online phone asks for everything after its cursor. Packets are never deleted from the queue during an incident, so a phone that comes online late still gets every alert that is still valid. Jobs carry a `to_device`; all phones relay them, only that phone shows them.

5.6 Storage and concurrency

- One process, one SQLite file in WAL mode (`PRAGMA journal_mode=WAL; synchronous=NORMAL; foreign_keys=ON`).
- All writes go through one lock, so there are no write conflicts; reads run freely.
- JSON fields (conditions, details, reasons, load) are stored as TEXT and parsed in Python.
- Times are ISO 8601 UTC strings; coordinates are WGS84 decimal degrees; distances use the haversine formula.

5.7 Caching

- **Snapshot cache:** the dashboard snapshot JSON is built once and reused until something marks it dirty, at most once per second.
- **Profiles and constants:** loaded once at start-up.
- **AI answers:** cached by text hash, so replays and duplicates cost nothing.
- **Phones:** profiles ship in the app's assets; shelters and zone names are cached at setup.

PART 6

Dashboard design

6.1 Layout

- **Top bar:** active hazard, offline banner, counters, buttons New alert, Intake, Paste, Teams, Replay, Chaos, Weights, Setup QR.
- **Main view:** map (60%) and ranked list (40%); clicking a case opens its explanation card; clicking a zone opens its zone card.
- **Teams and resources tab:** counters, team table, supplies table, warnings.
- **Bottom strip:** active alert, replies, overall coverage.

6.2 State and polling

One call every 2 s: `GET /api/snapshot` returns everything the screen needs (open cases with reasons, zones, teams, summary, active alerts, status). The app keeps it in one React state object; components only read from it. Commands (assign, alert, override, intake, move supplies) are POSTs; after a command the app fetches the snapshot immediately. If a call fails, the app keeps the last snapshot and shows "server not reachable" in the top bar.

6.3 Map layers

Layer	Source	Look
Base map	`/tiles/town.pmtiles` via protomaps-leaflet	Light, quiet colours
Zones	Snapshot zones (GeoJSON)	Outline; fill by coverage; red when Silent
Cases	Snapshot cases	Pins: red 70+, orange 40–69, yellow under 40; size by people
Teams	Snapshot teams	Icon by type, colour by status, grey when stale; line to the case; last 5 positions
Places	Snapshot shelters	Green labels with head count / capacity
Windows	Snapshot window markers	Orange markers with the closing time

6.4 Components

`App.jsx` (polling and state), `api.js` (fetch helpers), `TopBar`, `MapView`, `CaseList`, `CaseCard`, `ZoneCard`, `AlertComposer`, `IntakeForm`, `PasteBox`, `TeamsPanel`, `SuppliesTable`, `WeightsPanel`, `ChaosPanel`, `SetupQR`. Built with `npm run build` into `dashboard/dist`, which FastAPI serves at `/`.

PART 7

Data design

7.1 Wire format

On the wire, a packet has three fields. The signed fields travel as one JSON **string**, so Dart and Python sign and verify exactly the same bytes. `hops` is the only field outside the signature, because relays must change it; a forged hop count can only affect relaying, never content.

```
{
  "body": "{\"id\":\"8f1c2a7e-...\",\"kind\":\"report\", ... }",   // signed JSON text
  "sig": "base64 Ed25519 signature of the UTF-8 bytes of body",
  "hops": 2
}
```

Fields inside `body`:

Field	Example	Notes
`id`	8f1c2a7e-...	UUID v4, made by the sender
`kind`	report	See 7.2
`from_device`	dev_4KQ2 or command_post	Short device ID
`pubkey`	base64	Sender's Ed25519 public key
`created_at`, `expires_at`	2026-10-18T14:05:00Z	Phone time corrected by the offset from the setup QR; default expiry 24 h
`max_hops`	12	
`area`	tsm2k	Geohash, 5 characters (about a 5 km box); the only location outside the seal
`urgency`	0–4	Send order in the mesh
`replace_key`	ping:boat_2	Optional; relays keep only the newest packet with the same key
`to_device`	dev_9ZZ1	Optional; for jobs
`sealed` or `public`	base64 / object	Sealed payload (reports, replies, pings, acks) or public payload (alerts)

7.2 Packet kinds

Kind	Made by	Payload	Urgency	Sealed
`report`	Phone	hazard, for_whom, people, conditions, details, note, lat, lng, accuracy, landmark, role	0–3	Yes
`alert`	Server	alert_id, hazard, severity, zones, headline, do_now, places, issued_at, valid_until, replaces	4	No (signed)

Kind	Made by	Payload	Urgency	Sealed
`alert_reply`	Phone	alert_id, status (safe, need_help, helping), registry_id?, lat, lng	3 if need_help, else 1	Yes
`welfare_check`	Phone	registry_id or landmark, result (safe, needs_help), note	1–3	Yes
`volunteer`	Phone	name, skills, vehicle, lat, lng	1	Yes
`job`	Server	job_id, case summary, lat, lng, needs, first-aid tips, eta, place	3	No (signed)
`ack`	Phone	job_id, status (en_route, arrived, done, cant_reach, ready, out_of_service), outcome, reason	2	Yes
`team_ping`	Phone	team_id, lat, lng, accuracy, at, status, case_id, battery, fuel	1	Yes
`revoke`	Server	device IDs that are no longer trusted	4	No (signed)

Jobs are signed but not sealed so the team phone can read them without a private key exchange; they carry only what the team needs (no registry names). Outcome example: `{"people_helped": 1, "taken_to": "shelter_school", "supplies_used": {"oxygen_cylinder": 1}}`.

7.3 Hazard profiles

One JSON file per disaster in `shared/profiles/`, plus `common.json` for rules that apply to every disaster. Fields: `hazard`, `label`, `questions` (id, ask, type: yesno, choice, multi, options), `hours_rules`, `needs_rules`, `window`, `affected`, `keywords`, `supplies`, `tips`, `first_aid`, `alert_templates`. Rule tokens:

Token	Meaning
`x` / `!x`	Detail x is true / false (missing counts as false)
`x:a|b`	Detail x equals a or b, or (for multi) contains a or b
`cond:a|b`	The person's conditions contain a or b
`[]`	Always matches (use as the default rule)

A rule matches when all its tokens match. Hours = the smallest `hours` among matching rules in `common.json` and the case's profile; 24 if none. A full example is in Appendix B.

7.4 Server database (SQLite)

```
CREATE TABLE packets (
    id TEXT PRIMARY KEY, kind TEXT, from_device TEXT, created_at TEXT, received_at TEXT, hops
    INT, body TEXT, sig TEXT, result TEXT);
CREATE TABLE devices (
    id TEXT PRIMARY KEY, pubkey TEXT, role TEXT, trusted INT DEFAULT 0, revoked INT DEFAULT 0,
    first_seen TEXT, last_upload_at TEXT);
CREATE TABLE reports (
    id TEXT PRIMARY KEY, packet_id TEXT, case_id INT, source TEXT, hazard TEXT, for_whom TEXT,
    people INT, conditions TEXT, details TEXT, note TEXT, lat REAL, lng REAL, accuracy_m REAL,
    landmark TEXT, device_id TEXT, reporter_role TEXT, reader TEXT, reader_conf REAL, created_at
    TEXT, received_at TEXT);
CREATE TABLE cases (
```

```
    id INTEGER PRIMARY KEY, hazard TEXT, zone TEXT, lat REAL, lng REAL, accuracy_m REAL, people
    INT, conditions TEXT, details TEXT, hours_at_report REAL, reported_at TEXT, window_closes_at
    TEXT, points REAL, reasons TEXT, confidence REAL, needs TEXT, badges TEXT, status TEXT,
    team_id TEXT, shelter_id TEXT, source TEXT, verified INT DEFAULT 0, last_update TEXT,
    override TEXT);
CREATE TABLE alerts (
    id TEXT PRIMARY KEY, hazard TEXT, severity TEXT, zones TEXT, body TEXT, issued_at TEXT,
    valid_until TEXT, replaces TEXT, packet_id TEXT);
CREATE TABLE alert_replies (
    id TEXT PRIMARY KEY, alert_id TEXT, device_id TEXT, registry_id TEXT, status TEXT, lat REAL,
    lng REAL, at TEXT);
CREATE TABLE registry (
    id TEXT PRIMARY KEY, name TEXT, zone TEXT, lat REAL, lng REAL, conditions TEXT, phone TEXT,
    consent_at TEXT, confirmed_at TEXT, confirmed_by TEXT);
CREATE TABLE zones (
    name TEXT PRIMARY KEY, polygon TEXT, population INT, hard_hit INT, expected REAL, actual
    INT, registered INT, confirmed INT, silent INT);
CREATE TABLE teams (
    id TEXT PRIMARY KEY, name TEXT, type TEXT, capabilities TEXT, device_id TEXT, status TEXT,
    status_since TEXT, case_id INT, eta TEXT, lat REAL, lng REAL, ping_at TEXT, battery INT,
    fuel TEXT, load TEXT, flags TEXT, speed_kmh REAL);
CREATE TABLE team_pings (
    id INTEGER PRIMARY KEY, team_id TEXT, lat REAL, lng REAL, accuracy_m REAL, at TEXT, status
    TEXT, battery INT, fuel TEXT);
CREATE TABLE team_status_log (
    id INTEGER PRIMARY KEY, team_id TEXT, status TEXT, at TEXT, by TEXT, reason TEXT);
CREATE TABLE jobs (
    id TEXT PRIMARY KEY, case_id INT, team_id TEXT, created_at TEXT, eta TEXT, status TEXT,
    outcome TEXT, packet_id TEXT);
CREATE TABLE shelters (
    id TEXT PRIMARY KEY, name TEXT, type TEXT, lat REAL, lng REAL, capacity INT, occupancy INT,
    updated_at TEXT);
CREATE TABLE supplies (
    item TEXT, unit TEXT, place TEXT, qty REAL, warn_at REAL, PRIMARY KEY (item, place));
CREATE TABLE supply_moves (
    id INTEGER PRIMARY KEY, item TEXT, from_place TEXT, to_place TEXT, qty REAL, at TEXT, by
    TEXT, reason TEXT, job_id TEXT);
CREATE TABLE outbound (
    seq INTEGER PRIMARY KEY, packet_id TEXT, wire TEXT, to_device TEXT, created_at TEXT);
CREATE TABLE audit_log (
    id INTEGER PRIMARY KEY, at TEXT, actor TEXT, action TEXT, target TEXT, detail TEXT);
CREATE TABLE settings (
    key TEXT PRIMARY KEY, value TEXT);
-- settings holds weights, the active hazard and the AI status
CREATE INDEX ix_reports_case ON reports(case_id);
CREATE INDEX ix_cases_open   ON cases(status, points);
CREATE INDEX ix_pings_team   ON team_pings(team_id, at);
CREATE INDEX ix_replies      ON alert_replies(alert_id);
CREATE INDEX ix_registry     ON registry(zone);
```

7.5 Phone database (sqflite)

```
CREATE TABLE packets (
    id TEXT PRIMARY KEY, kind TEXT, wire TEXT, urgency INT, created_at TEXT, expires_at TEXT,
    hops INT, max_hops INT, replace_key TEXT, to_device TEXT, mine INT, delivered INT,
    received_at TEXT);
CREATE TABLE settings (
    key TEXT PRIMARY KEY, value TEXT);
CREATE TABLE alerts_seen (
    alert_id TEXT PRIMARY KEY, shown_at TEXT, reply TEXT);
-- settings: role, name, skills, device keys, server URLs, command post keys,
--           time offset, outbound cursor, mode, help_neighbours
CREATE INDEX ix_send_order ON packets(urgency DESC, created_at DESC);
```

PART 8

API

8.1 Conventions

- JSON over HTTP on the local network, base URL from the setup QR (for example `http://192.168.0.10:8000`).
- Errors: HTTP status plus `{"error": "code", "detail": "..."}`. Codes: `bad_signature`, `expired`, `rate_limited`, `unknown_kind`, `not_found`, `invalid`.
- Every dashboard command records an `audit_log` row. No login in the hackathon (one trusted laptop); see Part 15.

8.2 Endpoints

Method and path	Caller	Purpose
GET `/api/status`	Phones, dashboard	Health: up, AI reachable, packets last 5 min, last phone contact, scheduler last run
POST `/api/packets`	Online phones	Upload up to 100 wire packets; per-packet result
GET `/api/outbound?since=&device=`	Online phones	Packets after a cursor; new cursor
GET `/api/setup-qr`	Dashboard	PNG and JSON of the setup payload
GET `/api/profiles`	Dashboard, phones	All hazard profiles and the common rules
GET `/api/snapshot`	Dashboard every 2 s	Cases, zones, teams, summary, alerts, status in one response
GET `/api/cases`, `/api/zones`, `/api/teams`	Dashboard, tools	The same data as the snapshot, separately
POST `/api/alerts`	Dashboard	Create, update or cancel (all-clear) an alert
POST `/api/announcement`	Dashboard	Loudspeaker and radio script for an alert, per language
GET `/api/coverage`	Dashboard	Coverage per zone and the unconfirmed list
POST `/api/intake`	Dashboard	Report typed by a call-taker
POST `/api/paste`	Dashboard	Batch of free-text messages
GET `/api/suggest?case_id=`	Dashboard	Best team and place, alternatives, trade-off text
POST `/api/assign`	Dashboard	Assign a team to a case; creates a job packet
POST `/api/cases/{id}/override`, `/verify`, `/split`	Dashboard	Change status or points with a reason; verify; undo a merge
POST `/api/teams`, `/api/teams/{id}/status`	Dashboard	Add a team; set a status by hand with a reason
GET `/api/supplies`, POST `/api/supplies/move`	Dashboard, phones	Stock and run-out; restock, load, deliver, use, transfer

Method and path	Caller	Purpose
GET/POST `/api/shelters`	Dashboard	Places, capacity, head count
GET `/api/resources/summary`	Dashboard	Counters and warnings
POST `/api/registry/import`	Dashboard	Upload the registry CSV
POST `/api/settings/weights`, `/api/settings/hazard`	Dashboard	Change weights; change the active hazard
POST `/api/reports/simulate`, `/api/chaos/{kind}`	Replay, chaos panel	Demo tools

8.3 Examples

```
POST /api/packets
{"packets": [{"body": "{...}", "sig": "...", "hops": 2}]}
-> {"accepted": ["8f1c..."], "duplicate": ["a93b..."],
    "rejected": [{"id": "77d0...", "reason": "bad_signature"}]}

GET /api/outbound?since=41&device=dev_4KQ2
-> {"cursor": 44, "packets": [{"body": "{...alert...}", "sig": "...", "hops": 0}]}

POST /api/alerts
{"hazard": "heat", "severity": "warning", "zones": ["Ward 4", "Old Market"],
 "headline": {"en": "Extreme heat today, 12:00-17:00"},
 "do_now": {"en": ["...", "...", "..."]}, "places": ["cool_hall"],
 "valid_until": "2026-10-18T20:00:00Z", "replaces": null}
-> {"alert_id": "A-0007", "packet_id": "c1e2...", "outbound_seq": 45}

GET /api/suggest?case_id=142
-> {"best": {"team_id": "boat_2", "travel_min": 13, "place": "shelter_school"},
    "alternatives": [{"team_id": "boat_4", "travel_min": 21}],
    "tradeoff": "Boat 2 is also best for case #3. If it goes here, #3 waits about 35 more min"}

POST /api/assign
{"case_id": 142, "team_id": "boat_2", "place": "shelter_school", "by": "dispatcher"}
-> {"job_id": "J-031", "eta": "2026-10-18T15:46:00Z", "outbound_seq": 46}

GET /api/snapshot
-> {"rev": 812, "now": "...", "status": {...}, "alerts": [...],
    "summary": {"teams": {...}, "warnings": [...]},
    "cases": [{"id": 142, "points": 69, "reasons": [...], "confidence": 0.75,
               "badges": ["2 reports"], ...}],
    "zones": [{"name": "Ward 4", "silent": true, "expected": 24, "actual": 2,
               "coverage": 0.22, "polygon": {...}}],
    "teams": [{"id": "boat_2", "status": "on_scene", "lat": 0.0, "lng": 0.0,
               "ping_at": "...", "fuel": "half"}],
    "shelters": [...], "supplies": [...]}
```

PART 9

Algorithms

All constants come from Appendix A. All functions are pure and unit-tested (Part 13).

9.1 Hours left

```
def hours_left(case, now):
    rules = common.hours_rules + profile(case.hazard).hours_rules
    h = min([r.hours for r in rules if matches(r, case)], default=24)   # when the case changes
    return max(0, h - hours_between(case.reported_at, now))  # time passing raises urgency
```

9.2 Priority points

```
TIME_BANDS = [(1, 50), (3, 40), (6, 25), (12, 10), (inf, 0)]

def points(case, now):
    h = hours_left(case, now)
    time_pts = next(p for limit, p in TIME_BANDS if h <= limit)
    vuln = (W.cannot_walk * has(case, "cannot_walk|disabled")
            + W.elderly * has(case, "elderly")
            + W.child * has(case, "infant|child|pregnant")
            + W.alone * has(case, "alone"))
    closes = case.window_closes_at and hours_until(case.window_closes_at, now) <= WINDOW_HOURS
    window = W.window if closes else 0
    people = min(W.person * case.people, W.people_max)
    raw = time_pts + vuln + window + people
    total = raw * (TRUST_BASE + TRUST_SLOPE * case.confidence)
    return round(total), reasons(...)        # one {"text", "points"} per part that scored
```

9.3 Confidence

```
def confidence(case, reports, now):
    first = reports[0]
    c = START[first.source_type]                          # 0.9 team ... 0.3 silence
    c += MATCH_BONUS * (distinct_devices(reports) - 1)    # +0.15 per extra phone or source
    if best_accuracy(reports) < 50:
        c += GPS_BONUS                                    # +0.1
    elif only_landmark(case):
        c -= LANDMARK_PENALTY                             # -0.1
    c -= DECAY * floor(hours_between(case.last_update, now) / 2)    # -0.1 per 2 h
    if first.device_unknown and case.people > FAKE_PEOPLE:
        c = min(c, 0.3); add_badge(case, "verify")
    if case.verified:
        return 1.0
    return clamp(c, CONF_FLOOR, 1.0)
```

9.4 Merging

```
def attach(report):
    near = open_cases_near(report, MERGE_DISTANCE_M, MERGE_WINDOW_H)   # space + time filter
    for case in near:
        if hazard_compatible(case, report) and not contradicts(case, report):
            case.add(report)                    # people within 2; a shared word or condition
            case.conditions |= report.conditions
            case.details = newest_details(case.reports)
            case.location = most_accurate(case.reports)
            return case
    return new_case(report)
```

9.5 Silent Zones and warning coverage

```
def update_zones(now):
    hard = [z for z in zones if profile(active_hazard).affected(z)]
    for z in hard:
        others = [o for o in hard if o != z]
        rate = median(reports_per_100(o) for o in others) or DEFAULT_RATE
        z.expected = z.population / 100 * rate
        z.coverage = confirmed(z) / max(1, registered(z))
        quiet = z.expected >= SILENT_MIN_EXPECTED and z.actual / z.expected < SILENT_RATIO
        unheard = registered(z) >= COVERAGE_MIN_REGISTERED and z.coverage < COVERAGE_SILENT
        z.silent = quiet or unheard

def coverage_timeouts(now):
    for alert in active_alerts():
        wait = COVERAGE_TIMEOUT[alert.severity]           # emergency 30 min, warning 2 h
        if now - alert.issued_at < wait:
            continue
        for person in registry_in(alert.zones):
            if not confirmed_since(person, alert.issued_at):
                ensure_unconfirmed_case(person, confidence=0.3)
```

9.6 Suggest and trade-off

```
def suggest(case):
    capable = [t for t in teams
               if t.status == "available" and t.fuel != "low"
               and meets(t.capabilities, case.needs)
               and carries_or_can_fetch(t, case.supplies_needed)]
    ranked = sorted(capable, key=lambda t: travel_min(t, case))    # distance x 1.4 / speed
    best, tradeoff = ranked[0], None
    for other in top_open_cases(10):
        if other != case and best_for(other) == best:
            extra = travel_min(next_best(other), other) - travel_min(best, other)
            tradeoff = (f"{best.name} is also best for case #{other.rank}. If it goes here, "
                        f"#{other.rank} waits about {extra} more minutes")
    soon_free = [t for t in teams if t.status == "on_scene" and finishes_within(t, 10)]
    return best, ranked[1:3], tradeoff, soon_free
```

9.7 Team tracker

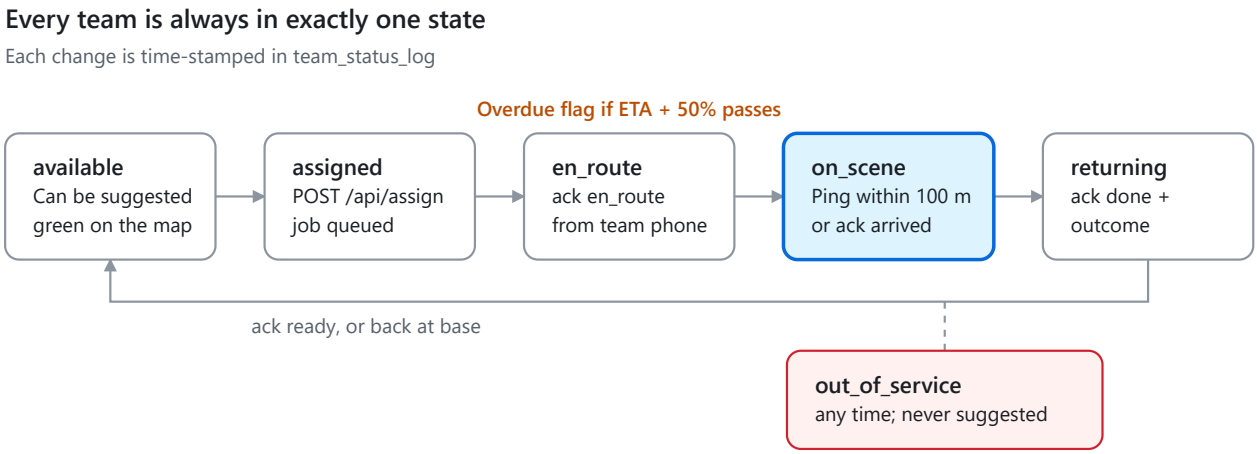


Figure 7. Team state machine. Only the server changes a team's state; phones send acks and pings that request changes.

```
def on_ping(p):
    if p.at > team.ping_at:                      # use the time the position was taken
        team.last_ping = p
    keep_last_positions(team, TRAIL)
    near = distance_m(p, team.case) <= ARRIVAL_RADIUS_M
    if team.status == "en_route" and near:
        set_status(team, "on_scene", by="ping")

def on_ack(a):          # en_route, arrived, done, cant_reach, ready, out_of_service
    set_status(team, status_for(a.status), by="team", reason=a.reason)
    if a.status == "done":
        apply_outcome(a.outcome)    # case done; shelter count up; used supplies leave the load

def team_checks(now):   # run by the scheduler
    for t in teams:
        late = now > t.eta + max(0.5 * t.travel, OVERDUE_MIN)
        if t.status in ("assigned", "en_route") and late:
            flag(t, "overdue")
        if t.status == "en_route" and now - t.ping_at > NOT_HEARD_MIN:
            flag(t, "not_heard_from")
```

9.8 Supplies and burn rate

```
left(item)        = sum of qty at every place         # command post + shelters + team loads
used_per_hour     = qty of moves with reason "used" in the last 3 h / 3
runs_out_in_hours = left(item) / used_per_hour         # none if nothing was used
low(item)         = left(item) <= warn_at(item)
```

PART 10

Security and privacy

10.1 Keys

Key	Type	Where	Used for
Command post signing key	Ed25519	`server/keys/cp_sign` (gitignored)	Signing alerts, jobs and revocations
Command post box key	X25519 (Curve25519)	`server/keys/cp_box`	Opening sealed reports, replies, pings and acks
Device signing key	Ed25519	Phone settings table	Signing every packet the phone makes

Keys are made once by `scripts/make_keys.py`. The setup QR carries `{"v": 1, "servers": ["http://192.168.0.10:8000", "https://...trycloudflare.com"], "sign_pk": "...", "box_pk": "...", "time": "...", "zones": [...]}`. A phone trusts only that signing key for alerts and jobs. Rescue-team devices are marked trusted on the dashboard after setup.

10.2 Signing and sealing

- **Sign:** `sig = Ed25519.sign(device_sk, utf8(body))`. Verify with the `pubkey` in the body; the server pins the first public key seen for each device ID.
- **Seal:** `sealed = base64(SealedBox(cp_box_pk).encrypt(utf8(json(payload))))`. Only the server can open it.
- **Alerts and jobs:** public or readable payload, signed with the command post signing key; phones verify against the key from the setup QR.

10.3 Threat model

Threat	Example	Defence
Fake alert	Someone broadcasts "dam broke, run"	Phones show only alerts signed by the command post key
Fake reports	A phone floods 100 reports	Signatures, 20 reports per device per hour, low starting trust, verify badge, a person confirms jobs
Reading private data	A relaying stranger opens a report	Sealed boxes; only geohash-5 and urgency are visible
Tampering	A relay edits a report	Signature over the whole body
Replay	An old packet is sent again	Unique IDs, server dedupe, expiry
Tracking people	Someone follows team pings	Pings sealed; replace_key limits copies
Stolen team phone	Device used to send false acks	Dispatcher marks the device revoked; a signed `revoke` packet spreads; server rejects it
Stolen laptop	Database and keys taken	Disk encryption; data deleted after the crisis; keys rotated

Lesson from Bridgefy (a mesh messaging app whose broken cryptography was published in 2020 and 2022): sign everything, seal private data, never invent our own crypto, and don't over-claim security. This is a hackathon prototype, not an audited system.

10.4 Data retention

Registry entries are opt-in with minimal fields. After an incident, `scripts/purge.py` deletes reports, registry data, pings and raw packets, keeping only anonymous counts for the after-action review.

PART 11

Reliability

11.1 Failure modes

Failure	Effect	Handling
No internet	AI reader unavailable	Keyword reader inside ingest; banner; AI refinement resumes when it returns
Towers down	Phones can't reach anything	Mesh store and forward; door knocks; QR backup
A phone loses Wi-Fi or data	Direct upload stops	After 2 failed checks it switches to the mesh; packets stay in the outbox until the server accepts them
Connection flickers	Phone flips between modes	Online after 1 good check, offline after 2 failures; sending twice is harmless because packets are deduplicated by ID
Server crash or restart	Dashboard blank for seconds	SQLite keeps everything; phones re-upload unacknowledged packets; dedupe makes it safe
Laptop dies	Command post down	Backups every 5 min; second laptop with a copy of keys and database takes over on the same IP
Bad data	Wrong ranking	Confidence, merge and split, overrides with reasons, audit log
Team silent	Unknown position	Grey icon after 10 min; not-heard-from flag after 15 min; manual status after a radio call
Phone storage full	Can't store more	Evict expired, then oldest low-urgency relayed packets; never own unacknowledged ones

11.2 Retries and idempotency

- Every packet has a unique ID; the server returns `duplicate` for repeats, so phones can always retry.
- Uploads and downloads retry with 10, 20, 30 s back-off; downloads use a cursor, so nothing is missed.
- Dashboard commands are idempotent where it matters: `assign` refuses if the case already has an active job unless forced with a reason.

11.3 Clocks

Phone clocks can be wrong. The setup QR carries the server time; phones store the offset and use it for `created_at`. The server also records `received_at`; if `created_at` is more than 10 minutes in the future it uses `received_at`. Expiry checks allow 1 hour of skew.

11.4 Backup and failover

`scheduler` copies the database every 5 minutes to `backups/` with the SQLite backup API; `scripts/backup.py` can also copy to a USB stick. A second laptop with the same keys can take over: restore the newest backup, start the server, give it the same IP address on the router.

11.5 Monitoring

- `/api/status` : uptime, packets last 5 min, rejected packets and reasons, AI status, last contact per device, scheduler last run, database size.
- The dashboard top bar shows these; a red dot appears when the scheduler hasn't run for 2 minutes.
- Logs: console plus `logs/server.log` ; every dispatcher action in `audit_log` .

PART 12

Scale and performance

12.1 Load estimate

Scenario	Load	Notes
Demo replay	500 messages in 2.5 min (about 3 per second); 10 teams pinging	Trivial for one laptop
Real town, 50,000 people, worst day	About 5,000 reports a day, peaks of 10 per second; 200 teams pinging every 2 min (under 2 per second); 3 dashboards polling	SQLite handles hundreds of writes per second
Mesh per phone	Up to 2,000 stored packets of about 1–2 KB; have lists under 100 KB	Fine for Nearby byte payloads in 30 KB batches

12.2 Targets

Measure	Target
Ingest per packet	Under 50 ms (keyword read, merge, score)
Snapshot build	Under 200 ms for 2,000 open cases; served from cache
Report from a phone with a connection	On the dashboard within 2 s
Alert: click to online phones	Within one outbound poll (10 s)
Dashboard refresh	Every 2 s
Scheduler run	Under 2 s every 30 s
Team phone battery	A full shift with pings on and a power bank

12.3 What changes at real scale

- Several command posts: each server is just another gateway; servers exchange packets with the same have/want protocol.
- Many dashboard users: WebSockets or server-sent events instead of polling; incremental snapshots by `rev`.
- Large areas: LoRa gateways (like the ClusterDuck Protocol) to bridge mesh islands; Postgres with PostGIS for spatial queries.

PART 13

Testing

13.1 Unit tests (pytest)

Test	Expect
Flood: water waist and rising	Hours left 2
Heat: confused and hot dry skin	Hours left 0.5
House 14 (oxygen, cannot walk, elderly, alone, 1 person, confidence 0.75)	69 points; 5 reasons
Heat case (heatstroke, elderly, alone, confidence 0.8)	66 points
Confidence: unknown phone + 1 matching phone + GPS 15 m	0.75
Merge: 2 reports 80 m apart, 30 min, same people	One case, "2 reports"
No merge: people 2 vs 9	Two cases
Silent: Ward 4 (2,000 people, median rate 1.2, 2 reports)	Expected 24, silent
Coverage timeout: warning, 2 h 1 min, unconfirmed person	Unconfirmed case at 0.3
Arrival: ping 80 m from the case while en route	on_scene
Overdue: ETA + 60%	Flag overdue
Outcome: 1 oxygen used	Oxygen left down by 1; move logged
Tampered body	Rejected: bad_signature
Same packet twice	Second is duplicate; nothing changes
Hours left after 2 h passes	Time points rise to the next band

13.2 Integration: replay tests

`scenario/replay.py --pack flood --speed 600 --check` replays the pack against a fresh server and asserts checkpoints: Ward 4 silent by 15:20; house 14 in the top 5; the stadium report has confidence 0.3 and a verify badge; after the internet cut every new report says "keywords"; Boat 3 flagged overdue by 16:00. The heat pack asserts that unconfirmed people become cases after 2 h and that the heatstroke case ranks first.

13.3 Device tests (real phones)

Test	Pass when
2 phones, hello world	Message shown both ways within 20 s
Direct when online	Phone on Wi-Fi: report on the dashboard within 2 s; mesh discovery is off
Fallback when offline	Wi-Fi off: the next report goes through a neighbour; Wi-Fi back on: the backlog uploads within 10 s
Mobile data only	Wi-Fi off, data on, public route running: report uploads directly

Test	Pass when
3 phones in a chain, airplane mode	A report reaches the server through 2 hops
100 packets sync	All transferred in under 60 s
Alert signed vs unsigned	Signed shows full screen; unsigned never shows
Server restart mid-sync	No packet lost; no duplicates on the dashboard
QR transfer	10 packets moved by one scan
Team ping and arrival	Walk 100 m: icon moves; on_scene within one ping
Screen off for 5 min	Note behaviour; keep screen on in the demo

13.4 Chaos tests

The chaos endpoints inject a duplicate, a fake report, a wrong location and an AI outage. Each must show the right badge or banner within one refresh.

13.5 Acceptance per gate

Gate	Hour	Pass when
Hello	2	Two phones swap a test message over Nearby
Walking skeleton	10	A report reaches the map through 2 phones with the internet off
Smart core	20	Flood replay ranks with readable cards, merges duplicates, shows a Silent Zone; the heat pack ranks heat cases
Loop back	28	Alert reaches an offline phone through 2 hops; reply updates coverage; job goes out, team shows on scene, done with an outcome updates supplies; overdue flag works; chaos handled
Trust and proof	33	Unsigned alert ignored; QR moves a message; simulation chart in the slides
Ready	36	5 clean demo runs in a row; backup video recorded

PART 14

Building plan

14.1 Repo layout

```
pulsetriage/
  README.md               how to run everything from a clean clone
  .gitignore              keys/ .env *.db backups/ .venv/ node_modules/ build/ dist/
  shared/
    constants.json        every number (Appendix A)
    profiles/             common.json flood.json heat.json earthquake.json ...
    examples/             one valid example per packet kind, an alert, a snapshot
  server/
    app/                  main config db crypto ingest profiles reader merge scoring zones
                          coverage alerts suggest teams supplies scheduler snapshot (.py)
    scripts/              make_keys reset_demo backup purge (.py)
    tests/                test_scoring test_merge test_zones test_coverage test_teams
                          test_supplies test_ingest test_contracts (.py)
    keys/ backups/ logs/  gitignored
    requirements.txt
    .env                  ANTHROPIC_API_KEY (gitignored)
  dashboard/
    src/                  App.jsx api.js components/*.jsx styles.css
    public/tiles/         town.pmtiles (offline map)
  field/
    lib/                  main.dart models/packet.dart services/*.dart screens/*.dart
    assets/profiles/      copied from shared/profiles at build time
    test/                 packet_test.dart (contract tests on shared/examples)
  scenario/
    zones.geojson  templates/  packs/flood/  packs/heat/  packs/earthquake/
    generate.py  replay.py  simulate.py
```

14.2 Hour 0 setup

Everyone: install Git, then:

```
git clone <repo-url> pulsetriage
cd pulsetriage
```

B, server (Windows commands):

```
python -m venv server\.venv
server\.venv\Scripts\activate
pip install fastapi "uvicorn[standard]" pydantic pynacl anthropic httpx
pip install shapely qrcode pillow pytest
pip freeze > server\requirements.txt
python server\scripts\make_keys.py
uvicorn app.main:app --host 0.0.0.0 --port 8000 --app-dir server

:: optional, when the laptop has internet: a public route for online phones
cloudflared tunnel --url http://localhost:8000

:: once, in an administrator prompt, so phones can reach the laptop:
netsh advfirewall firewall add rule name="PulseTriage 8000" ^
      dir=in action=allow protocol=TCP localport=8000
```

C, dashboard:

```
npm create vite@latest dashboard -- --template react
cd dashboard
npm install
npm install leaflet react-leaflet protomaps-leaflet
npm run dev            # during development (proxy /api to http://localhost:8000)
npm run build          # for the demo: FastAPI serves dashboard/dist at /
```

A, field app:

```
flutter create --org com.pulsetriage --platforms android field
cd field
flutter pub add nearby_connections permission_handler sqflite path geolocator
flutter pub add flutter_tts qr_flutter mobile_scanner pinenacl uuid http connectivity_plus
flutter run                          # with a phone connected by USB
flutter build apk --release
adb install build/app/outputs/flutter-apk/app-release.apk
```

D, data and AI:

```
pip install matplotlib shapely      (in the server venv)
python scenario\generate.py --pack flood
python scenario\replay.py --pack flood --speed 60
```

14.3 Contracts first

In hour 0 to 2, all four write `shared/examples/` : one valid JSON example for each packet kind, an alert, a profile and a snapshot. Server tests and Dart tests both load these files. If an example changes, everyone is told. This is what lets four people build in parallel without waiting for each other.

14.4 Phases and integration points

Hours	A: mobile	B: server	C: dashboard	D: data and AI	Gate
0–2	Flutter setup; Nearby hello	FastAPI skeleton; keys; firewall	Vite + map	Zones; API key test; examples	Hello

Hours	A: mobile	B: server	C: dashboard	D: data and AI	Gate
2–10	Report form from profile; outbox; direct upload; mesh fallback	`/packets` ingest (plain base64); cases; snapshot	Pins, list, card shell; intake form	flood.json, common.json; scenario v1; replay; prompt v1	Walking skeleton
10–20	Alert screen + TTS; alerts first; volunteer setup; reliability	Profile engine; points; confidence; merge; zones; registry	Card reasons; zones; hazard selector; banner; weights	Reader + fallback; heat.json; heat pack; flood v2; templates	Smart core
20–28	Outbound download; jobs; outcome form; pings; welfare	Alerts + coverage; suggest/assign; tracker; supplies; shelters; windows; chaos	Composer; Teams and resources; team icons; coverage; paste; chaos	Teams and supplies in packs; simulated pings; scripts; simulation start	Loop back
28–33	Signing and sealing; QR	Verify, unseal, sign; revoke	Polish	Earthquake profile; chart; slides	Trust and proof
33–36	Fixes	`reset_demo.py`; backup	Fixes	Demo script	Ready

Hour	Integration point	Who	How to test
2	Formats frozen in `shared/examples`	All	Contract tests pass on both sides
6	Phone uploads a plain report	A + B	`curl` the snapshot; report is there
8	Dashboard reads the snapshot	B + C	Pin appears within 2 s
14	Reader plugged into ingest	B + D	Paste a messy message; fields filled
22	Alerts and jobs flow out	A + B	Alert on an offline phone through 2 hops
26	Acks and pings flow in	A + B + C	Team icon moves; on_scene; supplies change
30	Crypto switched on	A + B	Tampered packet rejected; unsigned alert ignored

14.5 Build order per person

A: mobile

1. `packet.dart` + contract test on `shared/examples`.
2. `store.dart` (tables, priority order, replace_key).
3. `mesh_service.dart` hello, then have/want/packets.
4. Report form screen built from `flood.json`; save to the outbox.
5. `connection_service.dart`: online check, direct upload and download with a cursor; mesh only when offline.
6. Alert screen, TTS, reply buttons; alerts first.
7. Jobs screen, acks, outcome form; `location_service.dart` pings and arrival.
8. Welfare check, volunteer setup, QR backup; finally signing and sealing.

B: server

1. `db.py` schema (7.4); `config.py`; `/api/status`.
2. `ingest.py` with plain payloads; reports to cases; `/api/snapshot`.
3. `profiles.py`, `scoring.py`, `merge.py` with unit tests first.
4. `zones.py`, registry import, `coverage.py`.
5. `alerts.py`, outbound queue, `/api/outbound`.
6. `suggest.py`, `/api/assign`, `teams.py`, `supplies.py`, scheduler jobs.
7. `crypto.py` verify, unseal, sign; revoke; `reset_demo.py`, backups.

C: dashboard

1. `App.jsx` polling the snapshot; `MapView` with tiles and pins; `CaseList`.
2. `CaseCard` with reasons and buttons; `IntakeForm`.
3. Zones layer and `ZoneCard`; `TopBar` with banner and hazard selector; `WeightsPanel`.
4. `AlertComposer` with phone preview; coverage colours.
5. `TeamsPanel`, `SuppliesTable`, team icons and trails; suggest and assign flow.
6. `PasteBox`, `ChaosPanel`, `SetupQR`; projector polish.

D: data and AI

1. `zones.geojson`; `shared/examples` with the team.
2. `common.json`, `flood.json`; message templates; `generate.py` and `replay.py`.
3. `reader.py` keyword reader, then the AI reader with cache and timeout.
4. `heat.json` and the heat pack; registry CSV; alert and announcement templates.
5. Teams, supplies and simulated pings in the packs; `simulate.py` and the chart.
6. `earthquake.json`; replay checks (13.2); slides and demo script.

14.6 Definition of done

- Merged to `main`, and runs from a clean clone using the README steps.
- Passes its unit or contract tests; no disaster words hard-coded outside profiles.
- Errors are logged and shown, never a crash; works with the internet off.
- Shown to one teammate who didn't build it.

14.7 Git workflow

- `main` always runs. Each person works in their own folder and commits small, clear changes often.
- Pull before you push. Changes to `shared/` are announced to the team first.
- Tag every version that demos well: `git tag demo-v1`, `demo-v2`.
- Never commit keys, `.env`, databases or build folders.

14.8 Demo runbook

```
1. Router on; the laptop and the online phone (Phone C) join it. Note the laptop IP (ipconfig).
2. server\.venv\Scripts\activate
3. python server\scripts\reset_demo.py --pack flood
4. uvicorn app.main:app --host 0.0.0.0 --port 8000 --app-dir server
5. Open http://localhost:8000 full screen (125% zoom). Check the status bar is green.
6. Phones: app open, screen on, battery saver off.
   Relay phones: airplane mode, Bluetooth on, not joined to any Wi-Fi (so they stay offline).
7. One test alert and one test report end to end; then repeat step 3.
8. Live: python scenario\replay.py --pack flood --speed 60   (start when the script says so)
9. Heat switch: python server\scripts\reset_demo.py --pack heat --keep-server
                python scenario\replay.py --pack heat --speed 60
```

PART 15

What we'd revisit as it grows

Area	Today	Next
Command posts	One laptop	Server-to-server sync with the same packet protocol
Phones	Android only	iPhone app; bridging Android and iPhone meshes
Range	Bluetooth and Wi-Fi Direct	LoRa gateways to connect mesh islands
Dashboard	Polling, no login	Push updates; user accounts and roles; audit per user
Ranking	Hand-set weights	Weights validated with responders and real dispatch data
AI	Cloud model when online	Small on-device or laptop model for fully offline reading
Routing	Straight line × 1.4	Road routing with flooded or blocked roads removed
Warnings	App, door knocks, scripts	Cell broadcast, SMS gateway, voice calls, sirens integration
Keys	Files on disk	Android Keystore, key rotation, revocation lists at scale
Storage	SQLite	Postgres + PostGIS for larger areas and many users
Public route	Quick tunnel; address changes each run	Fixed tunnel or cloud relay; address changes pushed as signed packets
Accessibility	Read-aloud, icons	Testing with older and disabled users; sign-language and pictogram alerts

APPENDIX A

Constants

Saved as `shared/constants.json` and loaded by the server, the scenario tools and (where needed) the app. All are demo defaults.

Name	Value	Used in
`TIME_BANDS`	≤1 h: 50, ≤3 h: 40, ≤6 h: 25, ≤12 h: 10, else 0	Points
`W_CANNOT_WALK` / `W_ELDERLY` / `W_CHILD` / `W_ALONE`	15 / 10 / 10 / 10	Points
`W_WINDOW`, `WINDOW_HOURS`	20, 3	Points
`W_PERSON`, `W_PEOPLE_MAX`	2, 10	Points
`TRUST_BASE`, `TRUST_SLOPE`	0.6, 0.4	Points
`CONF_START`	team 0.9, call 0.8, registered 0.7, ai 0.6, unknown 0.5, keywords 0.4, silence 0.3	Confidence
`CONF_MATCH`, `CONF_GPS`, `CONF_LANDMARK`	+0.15, +0.1 (<50 m), −0.1	Confidence
`CONF_DECAY`, `CONF_FLOOR`, `RECHECK_H`	−0.1 per 2 h, 0.2, 3	Confidence
`FAKE_PEOPLE`, `RATE_LIMIT_PER_H`	10, 20	Fake guard
`MERGE_DISTANCE_M`, `MERGE_WINDOW_H`, `MERGE_PEOPLE_DIFF`	150, 3, 2	Merging
`SILENT_RATIO`, `SILENT_MIN_EXPECTED`	0.25, 8	Silent Zones
`COVERAGE_SILENT`, `COVERAGE_MIN_REGISTERED`	0.25, 10	Silent Zones
`ZONE_TASK_BASE`, `ZONE_TASK_PER_PERSON`, `ZONE_TASK_MAX`	60, 1, 90	Silent Zones
`COVERAGE_TIMEOUT`	emergency 30 min, warning 2 h	Coverage
`DETOUR`	1.4	Travel time
`SPEED_KMH`	boat 10, truck 25, medic 25, rescue 20, fire 25, welfare 20, foot 4	Travel time
`ARRIVAL_RADIUS_M`	100	Tracker
`OVERDUE_FACTOR`, `OVERDUE_MIN`	1.5, 10 min	Tracker
`NOT_HEARD_MIN`, `STALE_ICON_MIN`, `TRAIL`	15, 10, 5	Tracker
`PING_JOB_MIN`, `PING_IDLE_MIN`	2, 10	Field app
`BURN_WINDOW_H`	3	Supplies
`MAX_HOPS`, `EXPIRY_H`, `BATCH_KB`	12, 24, 30	Mesh
`RECONNECT_WAIT_S`, `UPLOAD_BATCH`, `OUTBOUND_POLL_S`	15, 100, 10	Mesh and sync

Name	Value	Used in
`CHECK_OFFLINE_S`, `CHECK_ONLINE_S`, `STATUS_TIMEOUT_S`	10, 30, 2	Connection modes
`ONLINE_AFTER_OK`, `OFFLINE_AFTER_FAILS`	1, 2	Connection modes
`STORE_MAX_PACKETS`	2000	Field app
`AI_TIMEOUT_S`, `AI_WORKERS`	4, 4	Reader
`DASHBOARD_POLL_S`, `SCHEDULER_S`, `BACKUP_MIN`	2, 30, 5	Server and dashboard
`QR_PACKETS`, `QR_CYCLE_S`	10, 2	QR backup
`CLOCK_SKEW_H`, `FUTURE_LIMIT_MIN`	1, 10	Clocks

APPENDIX B

Example profile: flood.json

Complete enough to build against. Tips are general public-safety wording; replace them with your national disaster agency's wording.

```
{
  "hazard": "flood",
  "label": {"en": "Flood"},
  "questions": [
    {"id": "water", "ask": "How deep is the water?", "type": "choice",
     "options": ["none", "ankle", "knee", "waist", "chest", "roof"]},
    {"id": "rising", "ask": "Is the water rising?", "type": "yesno"},
    {"id": "trapped", "ask": "Trapped (can't get out)?", "type": "yesno"}
  ],
  "hours_rules": [
    {"all": ["water:chest|roof"], "hours": 1, "text": "Water at chest or roof level"},
    {"all": ["water:waist", "rising"], "hours": 2, "text": "Water at waist and rising"},
    {"all": ["water:knee", "rising"], "hours": 4, "text": "Water at knee and rising"},
    {"all": ["trapped"], "hours": 6, "text": "Trapped"}
  ],
  "needs_rules": [
    {"all": ["water:waist|chest|roof"], "need": "boat"},
    {"all": ["trapped"], "need": "rescue"},
    {"all": [], "need": "truck"}
  ],
  "window": {"text": "Road floods", "source": "scenario_windows"},
  "affected": "flood_extent",
  "keywords": {
    "water:roof": ["roof", "on top of the house"],
    "water:chest": ["chest", "neck"],
    "water:waist": ["waist", "hip"],
    "water:knee": ["knee"],
    "rising": ["rising", "coming up", "getting higher"],
    "trapped": ["trapped", "stuck", "can't get out"]
  },
  "supplies": ["drinking_water", "life_jacket", "fuel", "blanket", "first_aid_kit"],
  "tips": {"en": [
    "Move people and valuables to the highest floor",
    "Do not walk or drive through moving water",
    "Switch off electricity if water is coming in and it is safe to do so"]},
  "first_aid": {"en": [
    "Keep the person warm and dry",
    "Do not give food or drink to someone who is unconscious"]},
  "alert_templates": {
    "warning": {"en": {
      "headline": "Flood warning for {zones}",
      "do_now": ["Move people and valuables upstairs",
                 "Check on elderly or ill neighbours",
                 "Do not walk or drive through floodwater"]}},
    "emergency": {"en": {
      "headline": "Flood emergency: {zones}. Act now.",
      "do_now": ["Go to the highest floor or roof now",
                 "Tap I need help if you cannot get out",
                 "Help neighbours who cannot move"]}}
  }
}
```

And the shared rules in `common.json`:

```
{
  "hours_rules": [
    {"all": ["cond:unconscious|bleeding"], "hours": 0.5, "text": "Unconscious or heavy bleeding"},
    {"all": ["cond:oxygen"], "hours": 3, "text": "Needs oxygen"},
    {"all": ["cond:power_dependent", "power_out"], "hours": 3,
     "text": "Medical device without power"},
    {"all": ["cond:injured"], "hours": 4, "text": "Injured"},
    {"all": ["cond:dialysis|insulin"], "hours": 12, "text": "Needs dialysis or insulin"},
    {"all": ["cond:no_food_water"], "hours": 12, "text": "No food or water"}
  ],
  "needs_rules": [
    {"all": ["cond:oxygen|bleeding|unconscious|injured"], "need": "medic"}
  ],
  "conditions": ["elderly", "child", "infant", "pregnant", "cannot_walk", "disabled",
                 "oxygen", "dialysis", "insulin", "power_dependent", "other_medicine",
                 "injured", "bleeding", "unconscious", "alone", "no_food_water"]
}
```